Deliverable 7: Security & Risk Assessment

What can go wrong, and what was deliberately left unsolved

Mills-Operation · AI Reliability Copilot for Hot Mill Operations

Security & Risk Assessment

This is not an attempt to solve every risk below in a five day prototype. Spotting them unprompted matters more here than closing every gap, and what follows is what was actually thought about while building this, including a couple only caught by going looking, not because they were obvious.

Prompt injection. The single stand copilot's prompt is built entirely from computed numbers, no free text field, so that part was mostly luck of the use case. The fleet wide ask the fleet text box is different, a supervisor can type anything into it, so it needed real design: the system prompt scopes the model to only these six stands and declines anything else (tested directly by asking it to write a poem), every tool it can call is read only, tool output is explicitly treated as data to report and never as instructions to obey, and the tool use loop is capped at 4 rounds so a confused model can't spin forever burning calls on one question.

Hallucination in decision support. This is the risk designed around most deliberately, because it is the one most likely to matter in practice. The prompt tells the model not to invent readings or history, but even with that constraint its interpretation of a correctly reported number was wrong once, a rolling mean bug made it say coolant pressure was up when it was actually crashing (full story in the AI partnership log). The real fix was not trusting the model more, it is that raw numbers are always shown next to the explanation on the dashboard, never replaced by it, so a supervisor never depends on the sentence being right.

Data leakage. Every copilot call sends sensor readings and anomaly scores to a third party API. That is data leaving the network by design and worth naming plainly rather than treating an external call as free. The data here is synthetic so there is nothing to actually leak, but in production that telemetry is operationally sensitive and would need a real data processing agreement, not an assumption that it is fine because it is just numbers. The API key itself lives in a gitignored local file, fine for a prototype, not a substitute for real secrets management in production.

Access control. Not implemented. No login, no separation between someone who can view an alert and someone with authority to act on it. That is a real gap, not an oversight being glossed over, and it needs solving before this touches anything with real operational consequence.

Audit trail. No persistent log right now of who saw which alert, when, or what they did about it. If a bearing fails after the tool flagged it, that question needs to be answerable from a log, not memory. Straightforward to add since every alert and every copilot call already has the data, it just is not written anywhere durable yet.

Model deprecation and drift. Two separate risks. The detector itself will drift as normal operation changes over time, and there is no retraining cadence or drift monitoring here. Separately, the copilot depends on a specific hosted model that could be deprecated or change behavior, the fallback path keeps the dashboard from hard breaking, but explanation quality would silently degrade and nothing currently alerts anyone that it happened.

Alert fatigue, the one that worries me most and is not on the standard checklist. Not a security risk in the traditional sense, but the one most likely to actually cause harm. Even a false alarm rate under 1% adds up into real noise across every stand and every shift, and a tool that gets ignored is worse than no tool at all because it creates false confidence that someone is watching. No complete solution here, per stand calibration and suppression logic would both help and neither is built, but it felt more worth naming plainly than most of the items above.

What was deliberately not built. The system explains and suggests, it never triggers a shutdown or pages anyone automatically. Keeping a human in the loop for every consequential action is the single biggest risk mitigation in this whole design, a decision made on purpose, not a limitation from running out of time.

Full detail on each of these is in `docs/security-risk.md` in the repository.